

WORK SPOT

토이 프로젝트

정태호

2026-01-19~

1. 도메인

(1) User 도메인

필드 : id(pk), name, nickname, job, gender(enum), ageGroup(enum)

비고) user는 db에서 예약어여서 @Table(name="users")로 테이블 명 변경

(2) Spot 도메인

필드 : id(pk), title, uniqueId, addrss, roadAddress, category, categoryGroupCode
CategoryGroupName, phone, mapx,mapy

비고) SpotSearchDto에서 Spot 엔티티로 변환할 때(toEntity()메서드)

Spot 생성자 필요 -> DTO 값을 엔티티로 옮길 때 사용

JPA 사용시 기본 생성자 무조건 필요! -> 매개변수가 있는 생성자 사용했으므로 기본 생성자 명시적으로 알려줘야함

(3) Agite 도메인

필드 : id(pk), customName, user(다대일 단방향), spot(다대일 단방향),
HasOutlet(enum)(콘센트 유무), memo

비고) 다대일 단방향 매핑, 지연로딩으로 설정(FechType.Lazy)

(4) Review 도메인

필드 : id(pk), title, user(다대일 단방향), spot(다대일 단방향),,
Content, createdAt(LocalDateTime)

2. 리포지토리

-> 리포지토리는 기본적으로 EntityManager를 필드로 가지고 시작

(1) UserRepository

- ① void **save**(User user) : persist
- ② User **findOne**(Long userId) : User 하나 찾아오기
- ③ List<User> **findAll**() : 모든 유저 찾아오기

(2) SpotRepository

- ① void **save**(Spot spot) : persist
- ② User **findOne**(Long spotId) : Spot 하나 찾아오기
- ③ List<Spot> **findAll**() : 모든 장소 찾아오기
- ④ Spot **findByUniqueid**(String uniqueid)

: 매개변수로 받은 uniqueid와 같은 장소를 받아온 후 return

비고) createQuery를 써서 getResultList 때문에 List로 받아지지만

Uniqueid는 유일한 컬럼이므로 비어있지 않다면 .get(0)으로 가져온다.

(3) AgiteRepository

- ① void **save**(Agite agite) : persist
- ② User **findOne**(Long agiteId) : Agite 하나 찾아오기
- ③ List<Agite> **findAll**() : 모든 아지트 찾아오기
- ④ **findAllByUserId**(Long userId) : userId로 검색
- ⑤ boolean **existsByUserIdAndSpotId**(Long userId, Long spotId)

: 이미 등록된 아지트인지 확인하는 로직

특정 유저(userId)가 이미 특정 장소(spotId)를 아지트로 등록했는지
안했는지 확인하는 로직 -> 이미 등록되어있다면 true 반환

⑥ void **deleteByUserIdAndSpotId**(Long userId, Long spotId)

: 특정 아지트를 DB에서 삭제시키는 로직

➔ 삭제쿼리 날림 : .excuteUpdate()

(4) ReviewRepository

① void **save**(Review review) : persist

② **findOne** : em.find(reviewId)

③ **findAll** : 모든 리뷰 찾아오기

④ **findByUserId** : userId 로 모든 리뷰 찾아오기

⑤ **findBySpotId** : spotId 로 모든 리뷰 찾아오기

3. Infrastructure (searchAPI)

사용한 SearchAPI

① NAVER 검색>지역 API

② KAKAO MAP>로컬 API

SearchApi 를 인터페이스로 구현하여 변경에 용이하게 함 -> @Primary 로 사용
구현체 설정

①에서 ②으로 바꾼 이유

: naver api 의 검색 개수가 5 개로 너무 한정적임

Kakao api 는 최대 검색 개수가 naver 에 비해 훨씬 널널함

SearchLocal 메서드 구현

```
public class KakaoApiClient implements SearchApi {  
  
    @Override 2 usages  
    public String searchLocal(String keyWord) {  
        try {  
            String openAPI = "https://dapi.kakao.com/v2/local/search/keyword.json?query=" +  
                URLEncoder.encode(keyWord, "UTF-8") + "&size=15";  
  
            URL url = new URL(openAPI);  
            HttpURLConnection con = (HttpURLConnection) url.openConnection();  
            con.setRequestMethod("GET");  
            con.setRequestProperty("Authorization", "KakaoAK " + [redacted]);  
            con.setRequestProperty("Content-Type", "application/json");  
  
            int responseCode = con.getResponseCode();  
            BufferedReader br;  
            if (responseCode == 200) {  
                br = new BufferedReader(new InputStreamReader(con.getInputStream(), "UTF-8"));  
            } else {  
                br = new BufferedReader(new InputStreamReader(con.getErrorStream()));  
            }  
  
            String inputLine;  
            StringBuffer response = new StringBuffer();  
            while ((inputLine = br.readLine()) != null) {  
                response.append(inputLine);  
            }  
            br.close();  
  
            return response.toString();  
        } catch (Exception e) {  
            throw new RuntimeException("카카오 API 호출 실패", e);  
        }  
    }  
}
```

㉠ openApi 의 url 정보를 가져옴

: URL 객체를 이용함

- ⑥ url 을 통해서 HttpURLConnection 객체를 만들
 : .setRequestProperty 를 통해서 인증 key 값을 전달
- ⑦ response message 에 보내주는 json 형식을 string 문자를 받아서 리턴함

➔ 서비스 계층에서 searchLocal 메서드 활용

4. 서비스

-> 서비스 계층에서는 기본적으로 @Transactional(readOnly = true)로 설정
 읽기 전용이 디폴트, 데이터 변경 메서드에만 transactional 붙이기

(1) UserService

필드 : userRepository

- ① Long **join**(User user) : userRepository.save(user)
- ② **findOne** : userRepository.findOne(userId)
- ③ **findAll** : userRepository.findAll()

(2) SpotService

필드 : SearchApi searchApi,

SpotRepository,

ObjectMapper : json 변환 라이브러리 (springboot 내장)

- ① List<SpotSearchDtoKakao> **searchKakao**(String keyword) : keyword는 검색어
 - (a) searchApi.**searchLocal**(keyword) 를 통해 String 형식의 json 받아옴
 - (b) **parseJsonKakao** 메서드를 통해 Documents로 받아옴

➔ ParseJsonKakao 는 objectMapper.readValue()로 KakaoSearchResponse 형식으로 리턴

➔ KakaoSearchResponse는 내부 클래스로 Document를 가지고 필드로 List<Document> documents를 가짐

(c) List에 담겨있는 Document들을 반복문을 돌며 dto로 변환 (from메서드)

(3) AgiteService

필드 : AgiteRepository

㉠ void **register**(Agite agite)

: AgiteRepository의 existsByUserIdAndSpotId를 호출 (이미 등록된 아지트인지 확인) -> 등록되지 않은 아지트라면 agiteRepository에 save함

㉡ void **cancel**(Agite agite)

: 등록되어 있는 아지트를 DB에서 삭제하는 로직

㉢ boolean **existsAgite**(Long userId, Long spotId)

: DB에 아지트가 있는지 확인하는 로직

(4) ReviewService

필드 : ReviewRepository reviewRepository

㉠ Long saveReview(ReviewForm form, User user, Spot spot)

: 등록 -> 매개변수로 들어온 form과 user, spot을 가지고 review객체를 만들어 save 함

㉡ Review findOne(Long reviewId) : reviewId를 가지고 리뷰 조회

㉔ List<Review> findAll() : 모든 리뷰 조회

㉕ List<Review> findAllSortByCreateTime()

: 모든 리뷰를 등록 시간에 맞게 sorting 한 후 리스트를 리턴

5. 컨트롤러

(1) UserController

필드 : UserService userService

㉑ **createForm** : 빈 UserForm을 생성해서 Post로 넘김 (create()로 넘어감)

➔ createUserForm.html

㉒ **create** : @ModelAttribute(생략되어있음)로 form을 받고 form으로 User객체 생성 후에 join

➔ PRG 패턴 적용 : redirect:/ 로 리턴하여 중복 등록 방지

㉓ **list** : model에 모든 User list를 담아서 넘김

➔ users.html -> 모든 user 조회

(2) SpotController

필드 : SpotService spotService

㉑ **searchForm** : 검색어 입력하는 화면 보여줌

[Home](#)

WORK SPOT

검색어

검색하기

© WORK SPOT V2

➔ 검색어를 입력 후 검색하기를 누르면 search 로 넘어감

⑥ **search** : @RequestParam으로 검색어를 매개변수로 받아옴

➔ SearchWord를 가지고 spotService.searchKakao 호출

➔ SpotSearchDto의 리스트로 담겨있는 검색 결과를 받은 후 Model에 넘김

➔ spotList.html에서 넘어온 List<SpotSearchDto>를 쪽 화면에 띄움

➔ 마지막 칸에 상세보기 버튼은 만듦. 상세보기를 누르기 전까지는 아직 DB로 넘어가지 않음. 상세보기 버튼을 누르면 POST로 /spot/detail 을 매핑하고 있는 spotDetail로 넘어감

[Home](#)

WORK SPOT

'성수동 카페' 검색 결과

장소명	주소	카테고리	상세보기
로우키퍼스탠드	서울 성동구 왕십리로4길 28-2	음식점 > 카페	상세보기
카모플라주	서울 성동구 아차산로1길 9	음식점 > 카페	상세보기
디어벌스데이	서울 성동구 서울숲길 55	음식점 > 카페 > 테마카페 > 디저트카페	상세보기
로우키퍼피 헤이그라운드점	서울 성동구 독성로1나길 5	음식점 > 카페 > 커피전문점	상세보기
구펠	서울 성동구 왕십리로14길 30-1	음식점 > 카페	상세보기
터틀힙 성수	서울 성동구 왕십리로14길 23	음식점 > 카페	상세보기
스타벅스 독성역	서울 성동구 아차산로 42	음식점 > 카페 > 커피전문점 > 스타벅스	상세보기
타크	서울 성동구 서울숲4길 16	음식점 > 카페	상세보기

© **spotDetail** : 상세보기를 눌렀을 때 dto가 entity로 변환되고 유에 저장됨
(spotService.getOrSaveSpot(dto))

- ➔ Spot 객체를 모델에 저장해서 spotDetail.html view 실행
- ➔ SpotDetail.html 에서는 장소의 상세정보 보여줌
- ➔ "내 아지트로 저장" 버튼을 누르면 agite/selectUser를 매핑한 AgiteController의 selectUser로 넘어감

장소 상세 정보

로우커피스탠드

주소
서울 성동구 왕십리로4길 28-2

카테고리
음식점 > 카페

고유 ID (Kakao)
940053669

좌표
X: 127.04663357436208 / Y: 37.54715716085294

내 아지트로 저장 **뒤로 가기**

3K SPOT V2

(3) AgiteController

필드 : AgiteService agiteService, UserService userService, SpotService spotService

㉠ **selectUser** : “내 아지트로 등록” 버튼을 눌렀을 때 어떤 유저의 아지트로 등록할 것인지 선택하는 페이지

➔ 버튼을 누르면 @RequestParam에 의해 hidden value로 spotId가 넘어옴

➔ 넘어온 spotId와 userList를 model에 담아서 넘김

➔ selectUser.html에서 선택한 user의 userId와 spotId를 hidden으로 넘기고 POST로 /agite/add로 매핑함

㉢ **addAgite** : agiteForm을 마저 채우고 agite를 DB에 등록하는 핸들러

➔ 먼저 이미 등록된 아지트인지를 agiteForm에 담겨있는 userId와 spotId를 가지고 확인함

➔ 이미 등록된 아지트라면 이미 등록된 아지트임을 보여주는 화면으로 리

턴

➔ 등록된 아지트가 아니라면 추가하는 view로 넘어감

㉔ **createAgite** : agiteForm 객체를 agite 객체로 넘겨서 DB에 저장

㉕ **chooseUser** : 아지트 조회를 할 때 user를 선택하는 화면

㉖ **list** : 선택한 user의 아지트 목록을 조회

(4) ReviewController

필드 : ReviewService reviewService, UserService userService, SpotService spotService

㉗ chooseUser : 리뷰작성을 눌렀을 때 작성할 user를 선택하는 페이지

㉘ searchWord : 장소를 검색하기 위해 검색어를 입력하는 페이지

➔ @PathVariable로 파라미터로 넘어온 userId를 model로 넘김

㉙ searchResult : 검색한 장소들이 나오고 리뷰 작성할 장소를 선택

➔ userId와 searchWord를 받아오고 model로 넘김

➔ 검색한 장소들은 아직 DB로 넘어가지 않음

㉚ chooseSpot

➔ 선택한 장소가 DB에 없다면 DB에 저장하고 spotId를 가져옴

➔ @PathVariable로 넘어온 userId와 spotId를 가지고 reviewForm을 만들고 model에 넘김

㉛ create : 리뷰제목과 내용을 입력하고 완성된 리뷰를 DB에 save 한다

➔ ReviewService.saveReview로 form을 review 객체로 변환 후 저장

➔ PRG 패턴 적용

➔ RedirectAttributes.addAttribute로 리다이렉트 시 파라미터를 넘겨서

home 화면에서 파라미터가 넘어오면 리뷰가 저장되었다는 문구를 출력할 수 있게 함

㉠ list : 모든 리뷰를 조회한다

- ➔ User의 닉네임으로 리뷰를 조회할 수 있는 기능 추가
- ➔ userNickName이라는 파라미터가 들어오면 그 nickName을 가지고 user를 알아낸 후 그 user가 작성한 리뷰를 조회해서 화면에 뿌림
- ➔ 모든 리뷰를 보여줄 때 작성한 시간 순서대로 정렬하여 보여줌

6. 더 구현 가능 한 기능들과 느낀점

- 카카오맵(로컬 외) API를 이용하여 장소 상세보기를 눌렀을 때 지도 화면을 보여 줄 수 있음
- Excel API를 이용하여 자신의 아지트 정보를 뽑아 낼 수 있게 함
- 아지트를 등록하거나 리뷰를 작성할 때 등록되어 있는 회원을 선택하는 것이 아닌 로그인 기능으로 구현함 -> 세션과 쿠키, Spring Security를 학습하여 구현
- html과 thymeleaf 부분은 ai의 도움을 많이 받았는데 thymeleaf 문법을 더 학습하여 직접 view를 구현 할 수 있어야함
- API를 사용할 때 변경에 용이 하도록 인터페이스 화 하여 구현하여야 함
 - ➔ NAVER API와 KAKAO API를 선택할 때 객체지향적으로 설계하지 않아서 변경에 어려움을 느꼈고, 중복된 코드 등을 인해 코드가 지저분해짐

느낌. 리팩토링 필요.